CSE 232
Computer Networks
Homework-4

Group-15

Anushk Kumar	2023115
Abhas Gupta	2023017
Arhan Jain	2023118
Ayush Kitnwat	2023160
Akshat Singh	2023064

Problem Statement

Your goal is to modify this implementation by supporting multiple clients. In this mode, the server accepts connection requests from multiple clients. Multiple clients can send the message to the server that it displays on STDOUT. After reading a message from STDIN, the server sends it to one of the clients of its choice (random is also fine). There is no need to change the client's implementation.

Implementation Overview

Thread Implementation

```c
void event_loop(int new_s, int client_num)
{
    fd_set readfds;
    struct timeval tv;
    int max_fd = STDIN_FILENO;
    int activity;
    char buf[MAX_LINE];

    if (new_s > max_fd) {
        max_fd = new_s;
    }
```

```
        if (FD_ISSET(new_s, &readfds)) {
            // data is available at client[i] connection
            int bytes = recv(new_s, buf, sizeof(buf), 0);
            buf[MAX_LINE-1] = '\0';
            if (bytes <= 0) {
                printf("client %i disconnected.\n", client_num);
                close(new_s);
                return;
            }
            printf("Client %i says: %s\n", client_num, buf);
            fflush(stdout);
        }
```

```
81
82  typedef struct {
83      int num;
84      int client_s;
85  } client_thread_args;
86
87  void client_handler(void *arguments){
88      client_thread_args *argu = (client_thread_args *)arguments;
89      event_loop(argu->client_s, argu->num);
90      free(argu);
91      pthread_exit(NULL);}
92
```

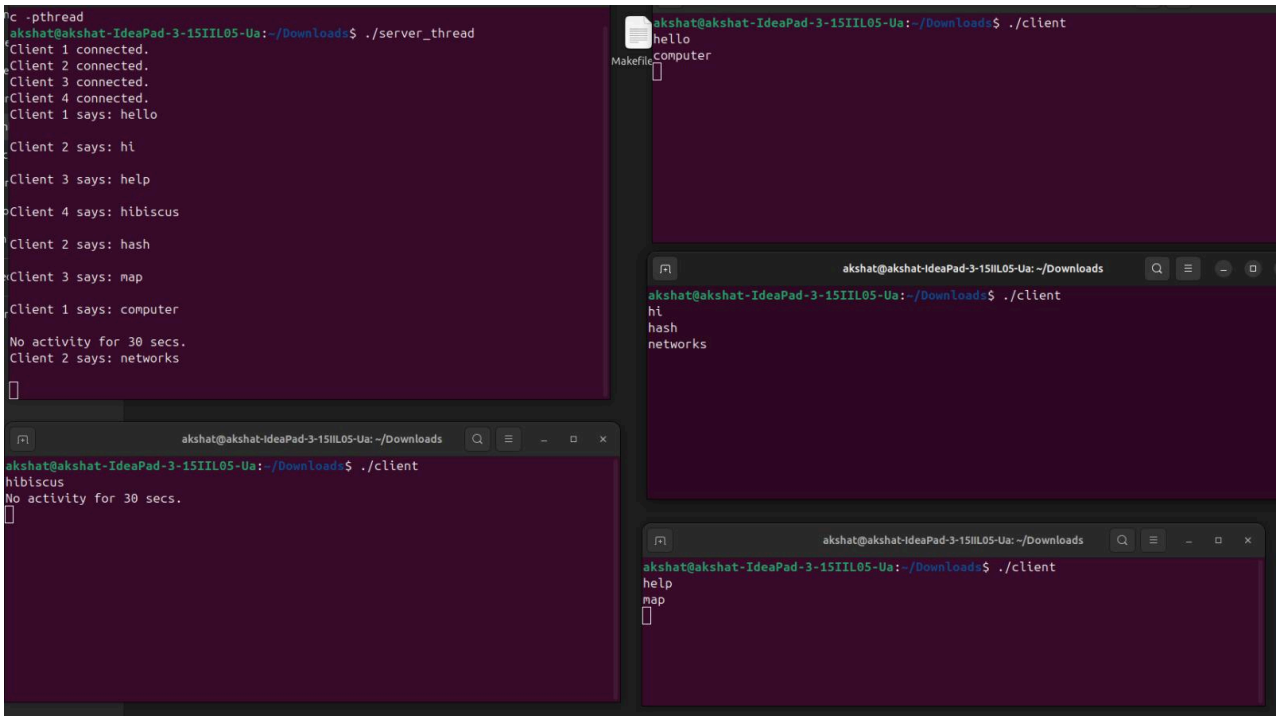
```
while(1) {
    int new_s;
    socklen_t addr_len;
    if ((new_s = accept(s, (struct sockaddr *)&sin, &addr_len)) < 0) {
        perror("accept failed");
        exit(1);}
    printf("Client %d connected.\n", client_num+1);

    client_thread_args *args = malloc(sizeof(client_thread_args));
    args->num = client_num+1;
    args->client_s = new_s;

    pthread_create(&clients[client_num++], NULL, (void *)client_handler, (void *)args);
}
```

We have used standard ``**pThreads**`` to implement parallel connection of multiple clients to the server using TCP protocol. The explanation of the changes are given below.

- ***ClientHandler function*** : It take input of arguments in format of struct client_thread_args and calls the main event_loop() and free the memory allocated by arguments and returns NULL for pthread_exit

- ***event_loop()*** : sets up file descriptors, select system call is used for check for activity (~30s). If data comes from STDIN then mutexlock is acquired to allow only one thread to read at a time. This is done infinitely until the client disconnects
- ***main()*** : sets up server configuration, sockets , ip address etc then it has an infinite loop which uses accept() sys call to accept new clients and make a struct type allocation of client_thread_args with clientNumber and socket. Creates a pthread which eventually calls event_loop for I/O operations.

Output



Yes, multiple clients can send messages on the server's terminal.

Event Based Implementation

For event based implementation we have utilized select system call in linux OS. Here is the detailed explanation of the same:

- ***event_loop() :*** Max clients we handled are 8 for which the currently connected clients are stored in the global array clients[ ]. We are using the listening socket in the event loop. Then whenever a new connection is to be established [i.e. when listening socket is ready to be read], we fill the first empty (0) client[i] with the fd of the client [which is stored into new_s]. Then we run a for loop through the client array, seeing if any client fd is ready to be read using FD_ISSET similar to the listening socket, and then the functionality is the same as it is for single client. When server needs to broadcast message, we go through the client array and the first client[i] which is non empty (i.e. not 0), we send the message to that client only. When a client wants to disconnect, we reset their fd to 0 in the client array.

- ***main()*** : This hasn't been much changed. Initialization of global clients[ ] and listening socket is done here. It still sets up server configuration, but only till the listening socket and initializes the listening socket in the global scope, and then calls the event_loop(). We are not doing accept in main as accept blocks until a single connection is established. On return, it closes all sockets and exits.

- We are using accept() and listening socket **only in the event loop to handle new incoming clients**.

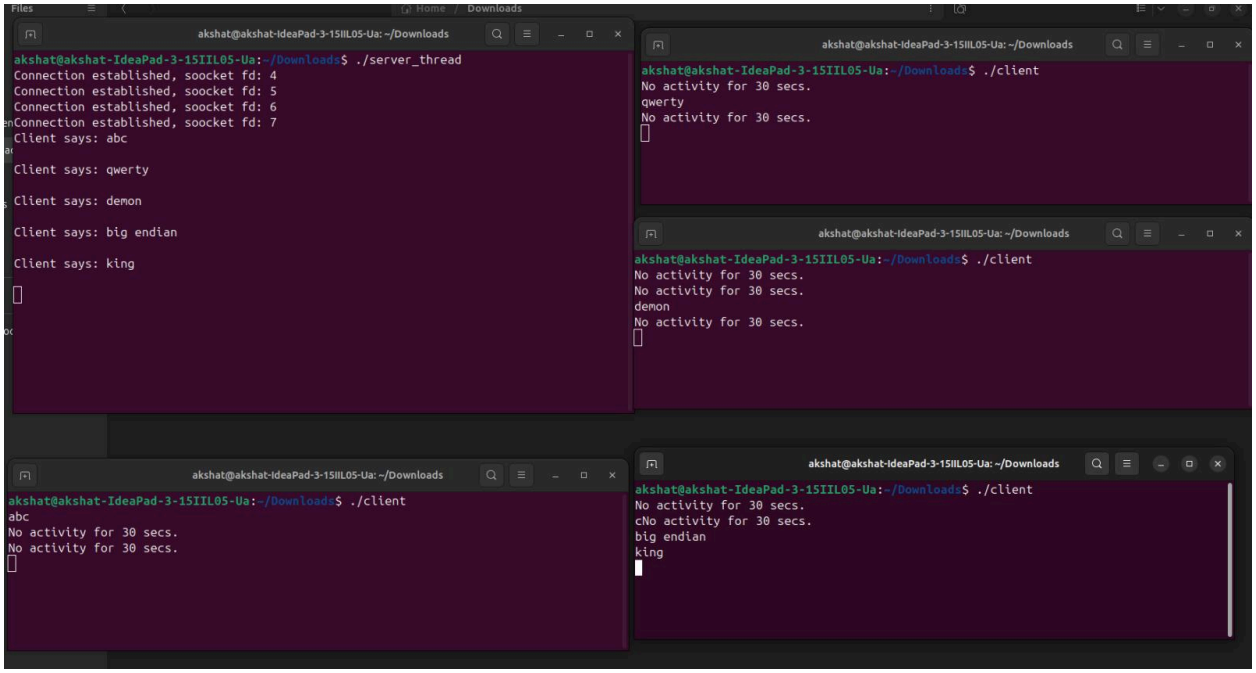
```
void event_loop(int new_s)
{
   fd_set readfds;
   struct timeval tv;
   int max_fd = STDIN_FILENO;
   int activity;
   char buf[MAX_LINE];
   struct sockaddr_in sin;
   socklen_t addr_len = sizeof(sin);
```

```
  if (listening_socket > max_fd) {
      max_fd = listening_socket;
  }

  while(1) {
      FD_ZERO(&readfds);
      FD_SET(STDIN_FILENO, &readfds);
      FD_SET(listening_socket, &readfds);

      for(int i = 0; i < 8; i++){
          if(clients[i]>0){
              FD_SET(clients[i], &readfds);
          }
          if(clients[i] > max_fd){
              max_fd = clients[i];
          }
      }


      tv.tv_sec = 30;
      tv.tv_usec = 0;

      activity = select(max_fd + 1, &readfds, NULL, NULL, &tv);

      if (activity < 0) {
          perror("select");
          break;
      } else if (activity == 0) {
          printf("No activity for 30 secs.\n");
          continue;
      }

      if (FD_ISSET(listening_socket, &readfds)) {
          if ((new_s = accept(listening_socket, (struct sockaddr *)&sin, &addr_len))
< 0) {
              perror("accept failed");
              exit(1);
          }
          for(int i = 0; i < 8; i++){
              if(clients[i]==0){
                  clients[i] = new_s;
```

```
                active_clients++;
                printf("Connection established, soocket fd: %d\n",clients[i]);
                break;
            }
        }
        if(active_clients>=8){
            close(new_s);
        }
    }
    for(int i = 0; i < 8; i++){
        // data is available at client[i] connection
        if(clients[i]>0 && FD_ISSET(clients[i], &readfds)){
            int bytes = recv(clients[i], buf, sizeof(buf), 0);
            buf[MAX_LINE-1] = '\0';
            if (bytes <= 0) {
                printf("client disconnected.\n");
                close(clients[i]);
                clients[i] = 0;
                active_clients--;
            }else{

                printf("Client says: %s\n", buf);
            }
        }
    }

    if  (FD_ISSET(STDIN_FILENO, &readfds)) {
        // data is avaliable at stdin
        if (fgets(buf, sizeof(buf), stdin) != NULL) {
            buf[MAX_LINE-1] = '\0';

            // broadcast message to all clients
            for(int i = 0; i < 8; i++){
                if(clients[i]!=0){
                    if (send(clients[i], buf, strlen(buf)+1, 0) < 0) {
                        perror("send error");
                        close(clients[i]);
                        return;
                    }
                    break;
                }
            }
```

```
            }
        }
    }
    for(int i = 0; i < 8; i++){
        close(clients[i]);
    }
}
```

Output



Yes, multiple clients can send messages on the server's terminal.